



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Laboratorio #5: File descriptors y durabilidad (Nginx 502 · MySQL fsync)

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

20 de octubre de 2026

2do Semestre 2026

De qué se trata el laboratorio

El laboratorio parte de dos tickets de producción. El #1043 dice que Nginx responde 502 y que el error.log trae "24: Too many open files", con la CPU al 5 % y la red normal. El #1044 dice que MySQL perdió transacciones que la aplicación daba por confirmadas después de un reinicio duro. Hay que resolver cada caso por separado y dejar escrito el camino hipótesis → comando → resultado → decisión → acción.

Todo se hizo dentro de la máquina virtual del curso (vm-sistemas-operativos, Ubuntu 18.04, kernel 4.15) y no en Docker, porque los experimentos tocan estado global del kernel o reinician la máquina. Nginx y MySQL estaban instalados pero detenidos, así que para reproducir los síntomas usé procesos de prueba equivalentes, como indica el enunciado. Nginx solo lo encendí al final para comprobar el límite de descriptores con systemd. Cada captura sale de una ventana de Windows Terminal conectada por ssh a la VM, y mi nombre aparece en el prompt.

Preparación del entorno

Paso 1. Entrar a la VM y revisar los límites de partida

Con la VM encendida (vagrant up) abrí una sesión ssh, creé la carpeta ~/laboratorios/lab5 y miré el kernel y los dos límites de descriptores de mi shell. El soft limit es 1024 y el hard limit 1 048 576. Ese 1024 es el techo con el que arrancan casi todos los procesos de esta VM, y es el número que va a aparecer en el 502.

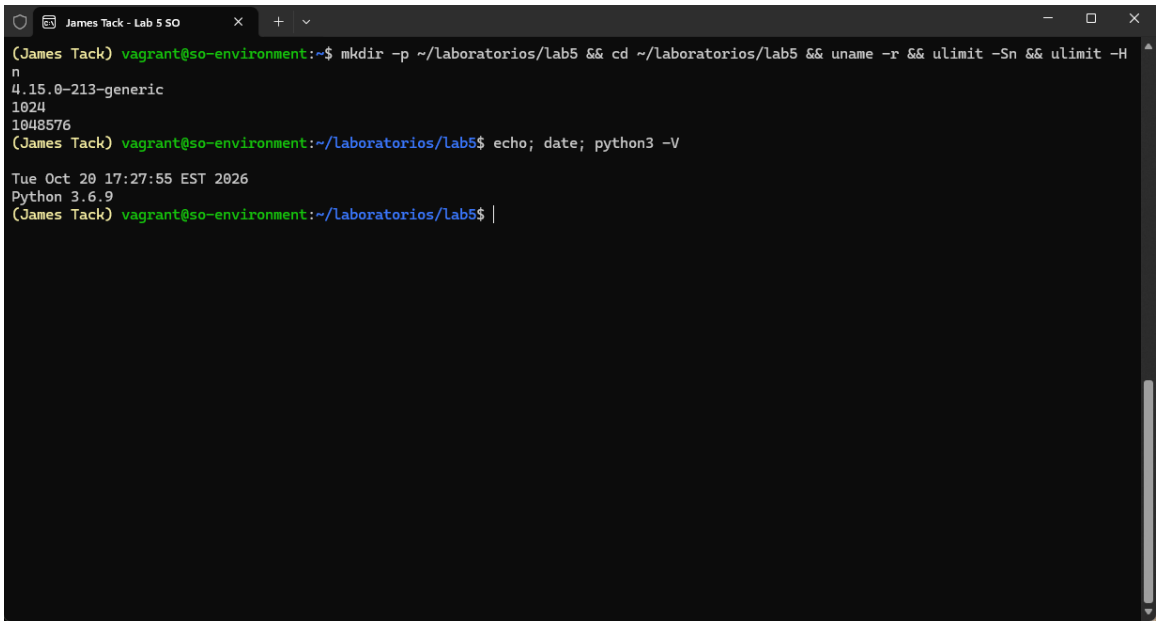


Figura 1. Carpeta del laboratorio, kernel 4.15, soft limit 1024 y hard limit 1048576 del shell.

N1. Reproducir el 502 y arreglarlo

Las cuatro decisiones de este nivel siguen el orden del enunciado. Antes de cada comando escribí la hipótesis, y después de cada resultado anoté la decisión.

P1. ¿Cuántos FDs usa el proceso y cuál es su techo?

Hipótesis. El límite que importa es el del proceso (RLIMIT_NOFILE), no el global del kernel. Un proceso sano debería usar muy pocos descriptores y estar muy lejos de ambos techos.

Lancé un proceso de prueba, python3 con un sleep de 600 s, guardé su PID en $P y consulté cuatro cosas: cuántas entradas tiene /proc/$P/fd, la línea "Max open files" de /proc/$P/limits, y los contadores globales fs.file-nr y fs.file-max.

```
python3 -c 'import time; time.sleep(600)' &
P=$!
ls /proc/$P/fd | wc -l
grep 'open files' /proc/$P/limits
cat /proc/sys/fs/file-nr
cat /proc/sys/fs/file-max
```

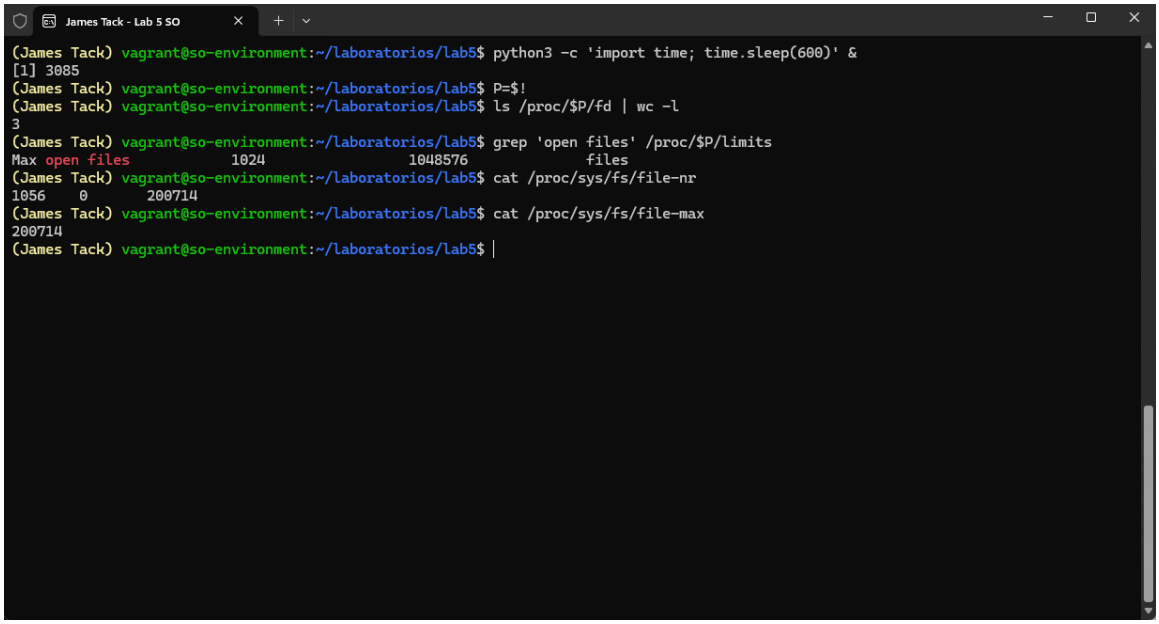


Figura 2. P1: el proceso usa 3 FDs con soft limit 1024; el sistema tiene 1056 FDs asignados de un máximo de 200714.

Resultado. El proceso usa 3 descriptores (0, 1 y 2) frente a un soft limit de 1024 y un hard limit de 1 048 576. En el sistema hay 1056 descriptores asignados, 0 libres y un máximo de 200 714 en fs.file-max. **Decisión.** Ni el proceso ni el kernel están cerca de su techo, así que el estado base es sano. Si los FDs se acercan a Max open files, el problema es del proceso (EMFILE, el error 24 del ticket). Si se acercan a file-max, es global (ENFILE). Son incidentes distintos. **Acción.** Medir cómo crece el proceso, que es lo que hace P2.

P2. ¿Por qué el proceso se queda sin descriptores?

Hipótesis. Si el proceso abre archivos sin cerrarlos, va a fallar justo cerca de su soft limit (1024), no cerca de file-max. Para provocarlo abrí /dev/null con el operador exec {fd}<> dentro de un subshell con el soft limit en 1024, hasta que bash devolviera error. Después pedí a Python los nombres de los errno 24 y 23.

```
( ulimit -Sn 1024; for i in $(seq 1 5000); do exec {fd}<>/dev/null || { echo "EMFILE en $i";
break; }; done; ls /proc/$BASHPID/fd | wc -l )
```

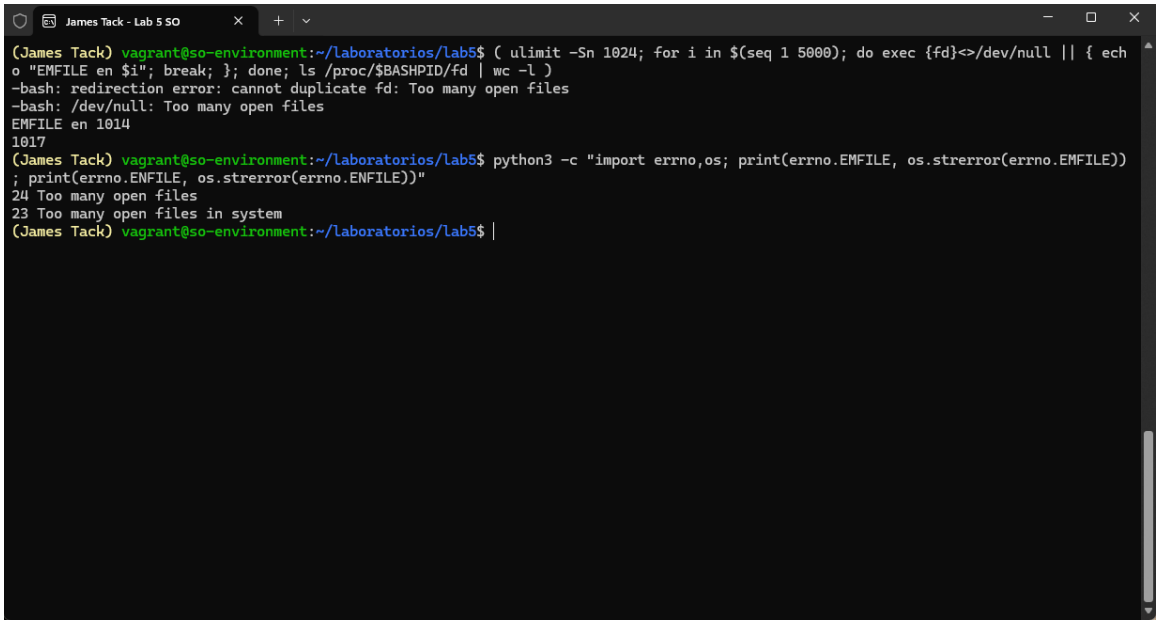


Figura 3. P2: bash falla en la apertura 1014 con "Too many open files"; errno 24 es EMFILE y errno 23 es ENFILE.

Resultado. El bucle se detuvo en la iteración 1014 con "Too many open files" y el subshell tenía 1017 descriptores abiertos (los tres estándar, el 255 de bash y los abiertos con exec). Python confirma los dos nombres: errno 24 = EMFILE, "Too many open files", y errno 23 = ENFILE, "Too many open files in system". **Conclusión.** Falla cerca de 1024, que es mi soft limit, así que el 24 del ticket es EMFILE y no una caída de red. La causa está en el proceso, no en el kernel (en P1 el sistema iba por 1056 de 200 714).

P3. ¿El código abre sin cerrar?

Hipótesis. El proceso llama a openat() muchas veces y casi nunca a close(). Escribí en una línea un proceso que abre /etc/hosts en bucle sin cerrarlo y le adjunté strace -c para contar las llamadas al sistema. Usé sudo porque strace necesita permiso de ptrace.

```
python3 -c 'import os,time
while True:
    try: os.open("/etc/hosts",0); time.sleep(0.02)
    except OSError: break' &
P=$!
ls /proc/$P/fd | wc -l
sudo strace -f -c -p $P 2>&1 | grep -E 'openat|close'
```

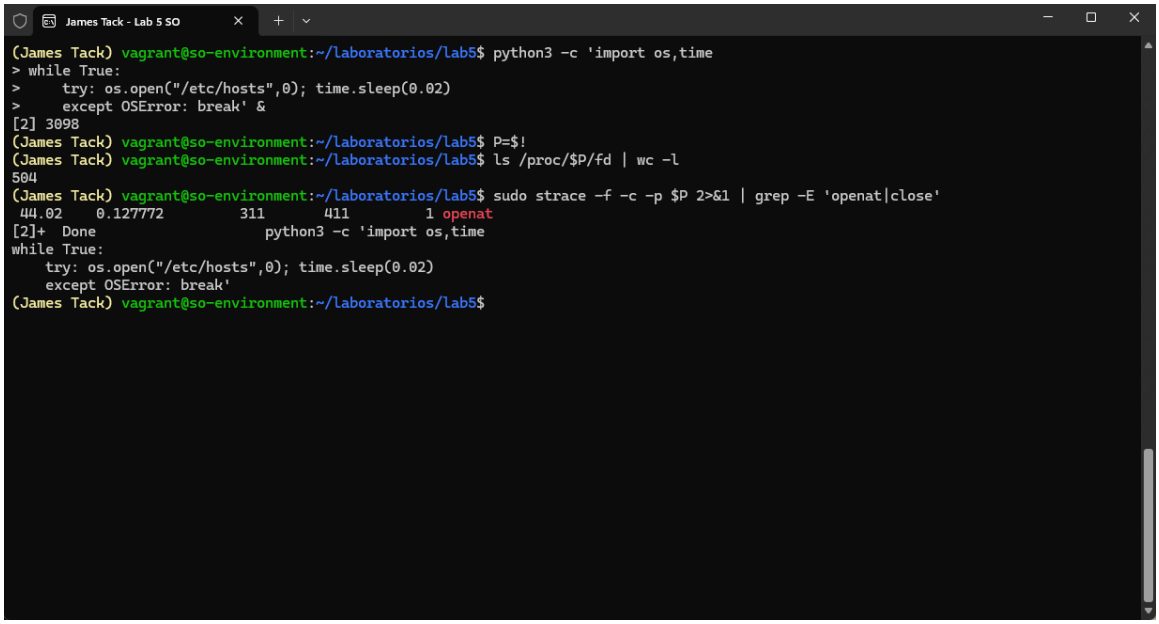


Figura 4. P3: strace -c solo muestra openat (411 llamadas, 1 con error); no aparece ninguna línea de close.

Resultado. Cuando consulté /proc/$P/fd había 504 descriptores. El resumen de strace trae 411 llamadas a openat con 1 error (la que chocó con el límite) y ninguna línea de close: el filtro grep -E con "openat|close" solo encontró openat. **Decisión.** openat ≫ close implica una fuga. En C sería un open() cuyo close() quedó fuera del finally. **Acción.** Aplicar la remediación y verificar (P4).

P4. Aplicar la remediación y verificar

Hipótesis. Con el límite elevado el mismo patrón ya no falla, y eso confirma que la causa era el techo del proceso. Subí el soft limit a 65535 dentro de un subshell (el hard limit de la VM es 1 048 576, así que se puede) y repetí 20 000 aperturas. Al final terminé el proceso de prueba del P1 con kill %1.

```
ulimit -Hn; ulimit -Sn
( ulimit -n 65535; ulimit -Sn; for i in $(seq 1 20000); do exec {fd}<>/dev/null; done; echo
"OK"; ls /proc/$BASHPID/fd | wc -l )
```

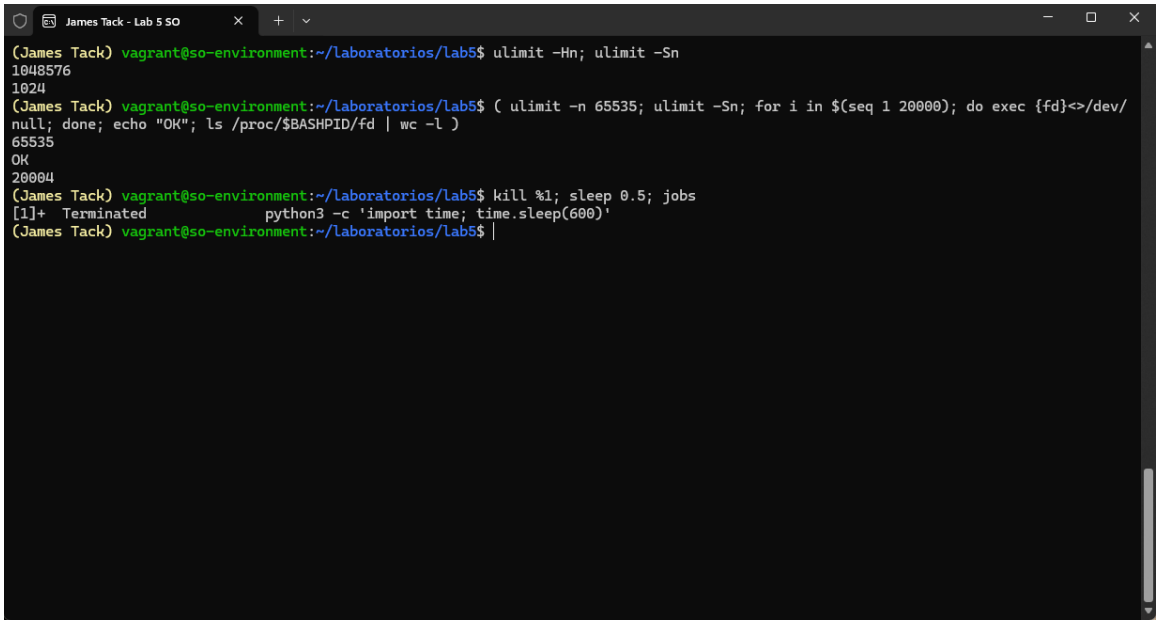


Figura 5. P4: con soft limit 65535 las 20 000 aperturas terminan en OK y el subshell llega a 20 004 FDs.

Resultado. Las 20 000 aperturas terminan sin error y el subshell queda con 20 004 descriptores abiertos. **Decisión.** Con el límite elevado ya no falla, por lo tanto la causa era el límite del proceso y no el kernel. **Acción.** Subir LimitNOFILE en systemd y cerrar los FDs en el código. Eso lo apliqué con servicios reales en N2.

Las cuatro decisiones y el runbook del 502

#	Hipótesis	Comando	Resultado	Decisión → acción
P1	El techo es del proceso, no del kernel	grep 'open files' /proc/$P/limits; cat /proc/sys/fs/file-nr	3 FDs de 1024; 1056 de 200714	Estado base sano → medir el crecimiento
P2	Falla cerca del soft limit	ulimit -Sn 1024 + exec {fd}<>/dev/null	EMFILE en la apertura 1014; errno 24 vs 23	Es EMFILE, no ENFILE → culpar al proceso
P3	Abre sin cerrar	strace -f -c -p $P	openat 411, close 0	Hay fuga → corregir el código
P4	Subir el límite lo evita	ulimit -n 65535 + 20000 aperturas	OK, 20004 FDs	Era el límite del proceso → LimitNOFILE

El runbook del incidente queda así. La primera fila es el caso del ticket #1043 y la segunda es el caso global que descartamos.

Síntoma	Comando	Lectura	Acción
502 + "24: Too many open files" en error.log	grep 'open files' /proc/<pid>/limits; ls /proc/<pid>/fd | wc -l	FDs usados ≈ Max open files (EMFILE, límite del proceso)	Subir LimitNOFILE en systemd y cerrar FDs en el código
Errores de apertura en varios servicios a la vez	cat /proc/sys/fs/file-nr	Asignados cerca de fs.file-max (ENFILE, agotamiento global)	Subir fs.file-max (no es este ticket)

N2. ¿Quién se queda sin FDs?

Escenario y método

Hay tres procesos "app" (app1.py, app2.py y app3.py), uno sano y dos con fuga, y no sé cuál es cuál. Tampoco hay receta ni lsof. Para que el escenario fuera completo levanté también fakedb.py, un servidor mínimo que escucha en el puerto 3306 y acepta conexiones sin responder, haciendo el papel de MySQL. Las tres apps se lanzan con su salida redirigida a /tmp.

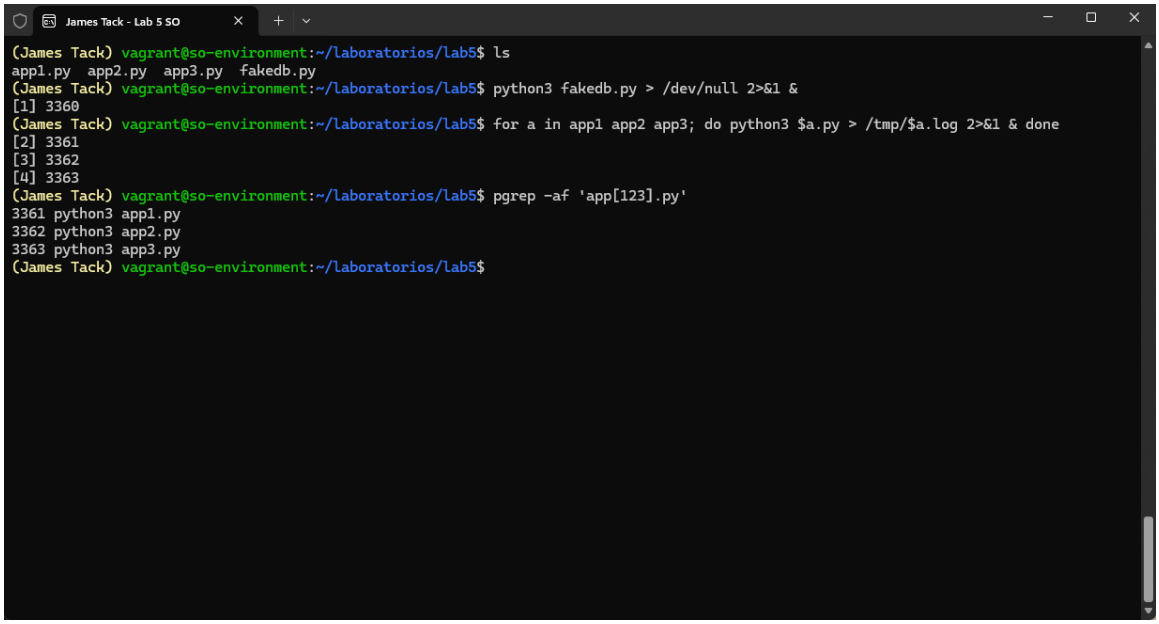


Figura 6. Archivos del escenario y lanzamiento de fakedb.py y de las tres apps.

Hipótesis previa. Una fuga se reconoce por tendencia, no por valor. Los dos culpables deberían tener un conteo de FDs que crece con el tiempo y el sano uno constante. Elegí /proc/<pid>/fd porque es la fuente que el kernel mantiene siempre y no necesita instalar nada. Medí dos veces con 8 segundos de diferencia.

```
for t in 1 2; do for p in $(pgrep -f 'app[123].py'); do echo "$(date +%T) pid=$p fds=$(ls
/proc/$p/fd | wc -l) $(ps -o args= -p $p)"; done; echo; sleep 8; done
```

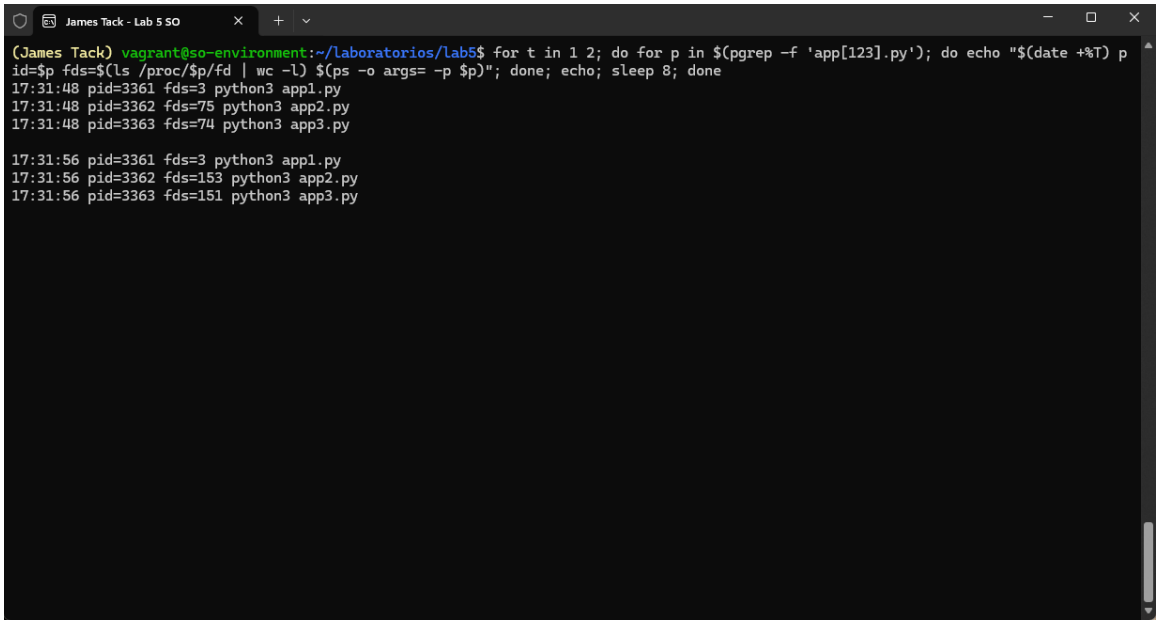


Figura 7. Dos mediciones separadas por 8 s: app1.py se queda en 3 FDs; app2.py y app3.py crecen unos 80 FDs.

Resultado. app1.py se mantiene en 3 descriptores. app2.py pasa de 75 a 153 y app3.py de 74 a 151 en 8 segundos, es decir, unos 10 FDs por segundo cada una. A ese ritmo las dos llegan a 1024 en menos de dos minutos y empiezan a fallar con EMFILE, que es la situación del ticket.

Qué recurso agota cada una. Para saber el tipo de descriptor listé /proc/<pid>/fd con ls -l, tomé el destino de cada enlace, cambié el número de inodo del socket por una N y conté. Aparte conté las conexiones establecidas hacia el puerto 3306 con ss.

```
for p in $(pgrep -f 'app[23].py'); do echo "== pid $p"; ls -l /proc/$p/fd | awk 'NR>1{print
$NF}' | sed -E 's/[0-9]+\]$/N]/' | sort | uniq -c | sort -rn | head -3; done
ss -tn state established '( dport = :3306 )' | wc -l
```



Figura 8. Tipo de FD por proceso: app2.py acumula archivos /etc/hosts y app3.py acumula sockets; ss muestra las conexiones a :3306.

Resultado. El proceso 3362 (app2.py) tiene 313 descriptores apuntando a /etc/hosts: son archivos. El proceso 3363 (app3.py) tiene 310 de tipo socket:[N], y ss cuenta 346 líneas (cabecera incluida) de conexiones establecidas hacia :3306, un número del mismo orden que los sockets del proceso, que seguían creciendo. Por eso el recurso agotado de app2.py son archivos y el de app3.py son sockets hacia :3306, el patrón típico de un pool de MySQL que no cierra conexiones.

Confirmación con strace. Por último adjunté strace -c a cada proceso durante 4 segundos con timeout -s INT, que le manda Ctrl+C para que imprima su resumen. Busqué las syscalls que crean descriptores y close, que los libera.

```
for p in $(pgrep -f 'app[23].py'); do echo "== pid $p"; sudo timeout -s INT 4 strace -c -f -p $p 2>&1 | grep -E 'openat|socket|connect|close'; done
```

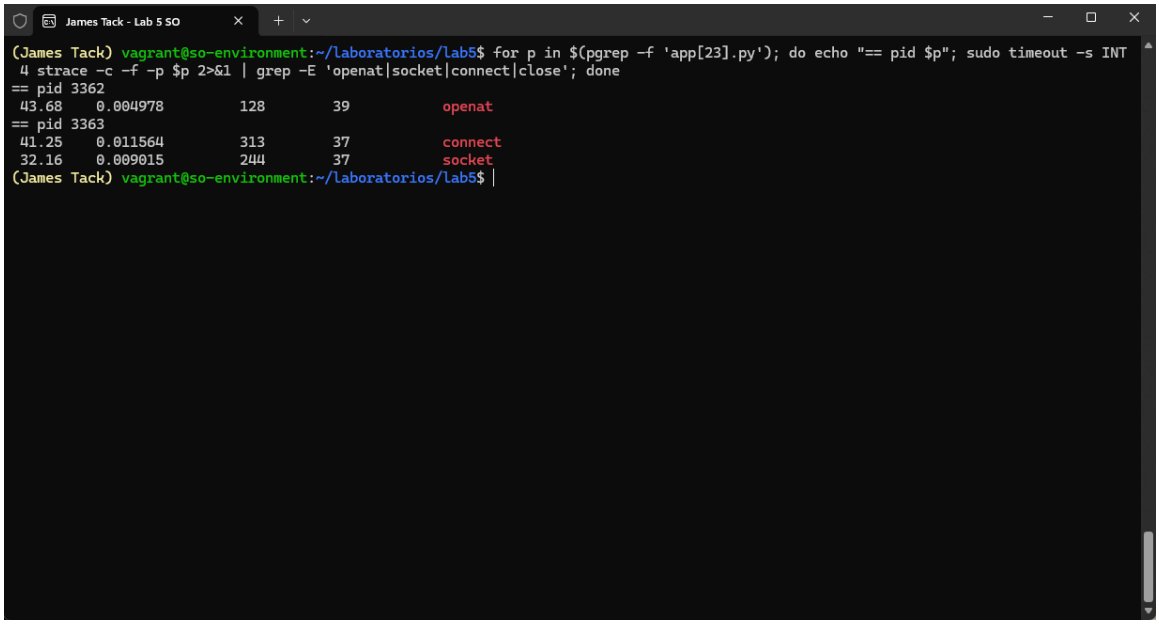


Figura 9. strace -c: app2.py hace openat sin close; app3.py hace socket y connect sin close.

Resultado. El proceso 3362 hizo 39 llamadas a openat y el 3363 hizo 37 llamadas a socket y 37 a connect. En ninguno apareció close. **Veredicto.** Los culpables son app2.py (fuga de archivos) y app3.py (fuga de sockets hacia :3306). app1.py es el sano.

Remediación en tres capas

Capa 1: código. Cada fuga se corrige liberando el descriptor siempre, también cuando hay error. En app2.py el os.open queda dentro de un try y el os.close en un finally. En app3.py la conexión se abre con with, que cierra el socket al salir, en vez de guardarla en una lista. La figura muestra el diff entre la versión con fuga y la corregida.

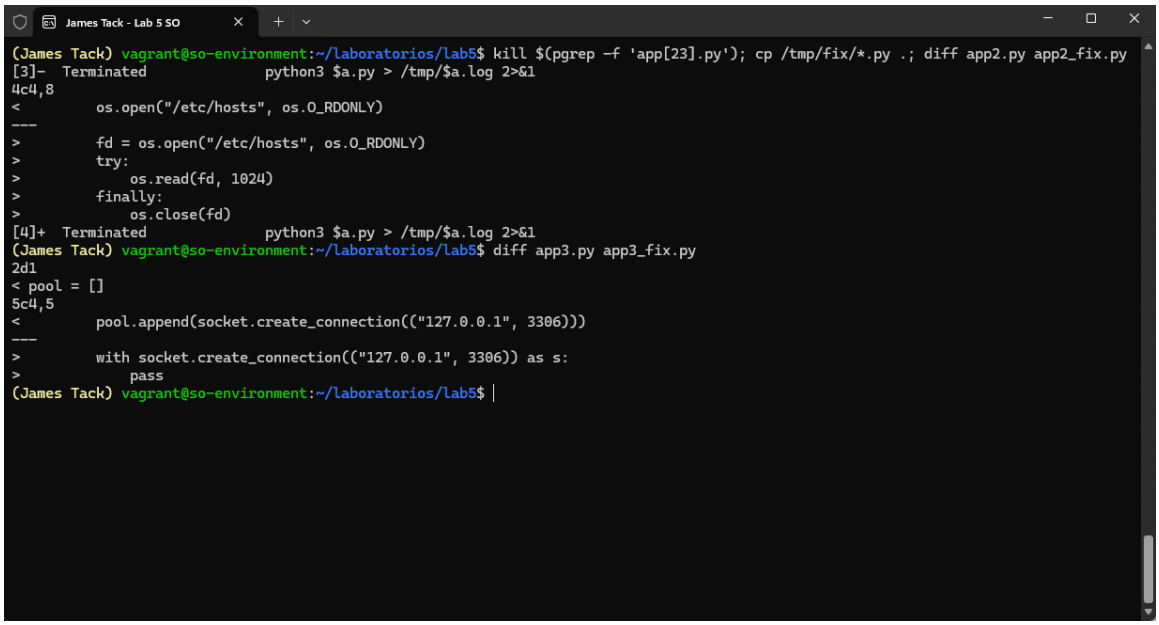


Figura 10. Capa de código: diff de app2.py y app3.py contra sus versiones corregidas.

Para verificar lancé las dos versiones corregidas y repetí el muestreo. Los descriptores se quedan en 3 en las dos mediciones, separadas por 8 segundos y tomadas después de 15 segundos de ejecución, en lugar de crecer 10 por segundo.

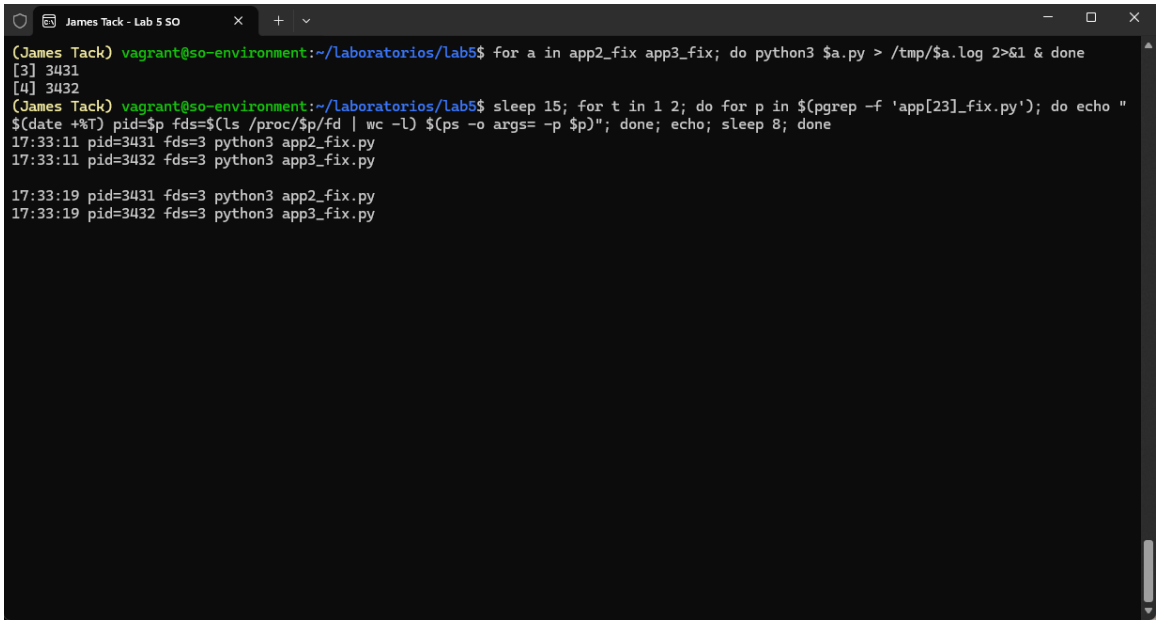


Figura 11. Con el código corregido app2_fix.py y app3_fix.py se mantienen en 3 FDs.

Capa 2: proceso. El código corregido evita la fuga, pero el techo sigue siendo bajo para un servicio que atiende muchas conexiones legítimas. Con Nginx real vi que systemd le da LimitNOFILE=4096 por defecto. Creé un drop-in en /etc/systemd/system/nginx.service.d/limits.conf con LimitNOFILE=65535, recargué systemd, reinicié el servicio y comprobé en /proc que el proceso hereda el límite nuevo.

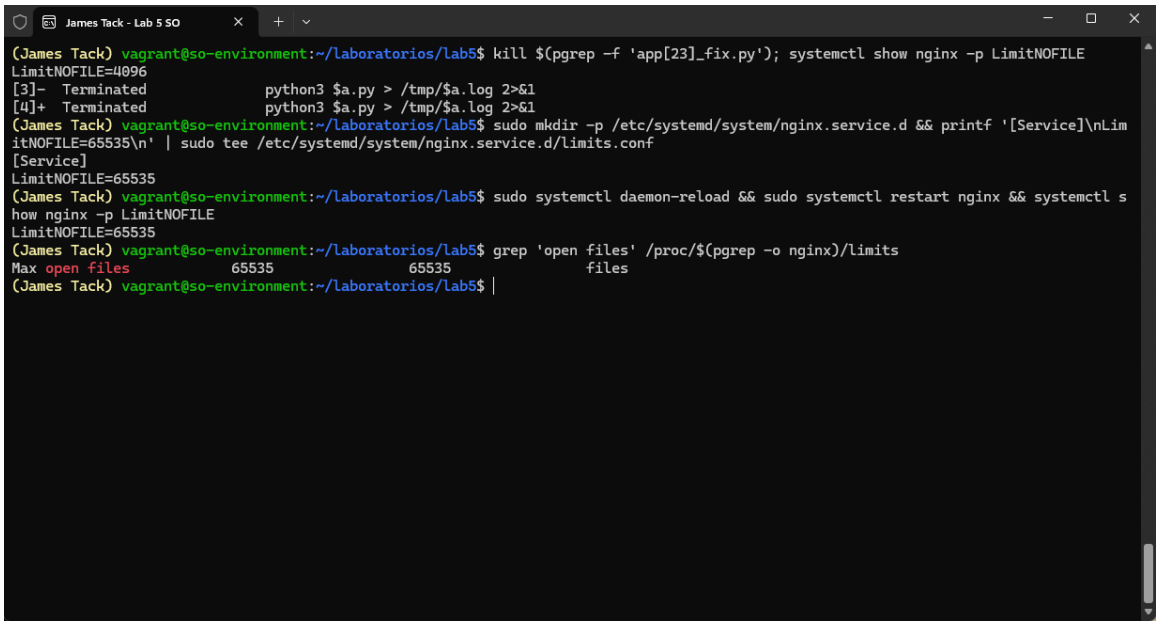


Figura 12. Capa de proceso: LimitNOFILE pasa de 4096 a 65535 y /proc/<pid>/limits de Nginx lo confirma.

Capa 3: kernel. El tope global es fs.file-max, que en esta VM vale 200 714 con apenas 2144 descriptores asignados. No hace falta subirlo: si sube LimitNOFILE de un servicio, hay que comprobar que la suma de lo que pueden abrir todos los servicios no supere file-max. Si hiciera falta, se deja persistente en /etc/sysctl.d con sysctl --system, como se ve abajo. El tope por proceso nunca puede pasar de fs.nr_open (1 048 576).

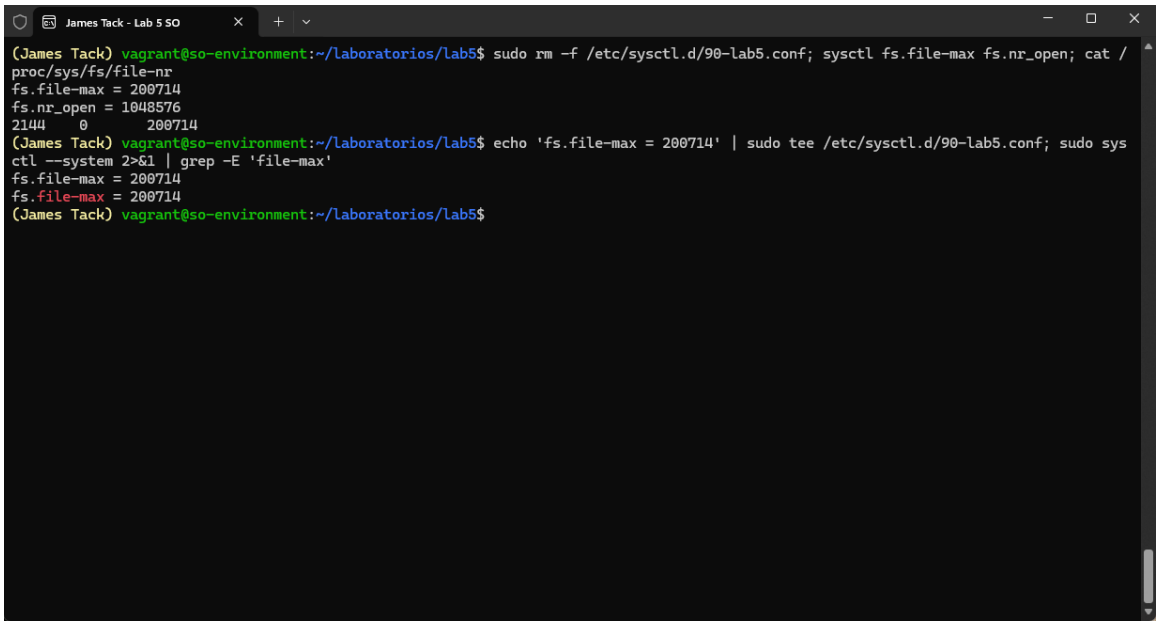


Figura 13. Capa de kernel: fs.file-max 200714, fs.nr_open 1048576, file-nr 2144 y la línea persistente en sysctl.d.

Esta es la razón de las tres capas. El código corrige la causa, el límite del proceso da margen y el kernel pone el techo para todos. Si falta alguna, el incidente vuelve por otro lado.

N3. "Se perdieron datos": write() contra fsync

P1. ¿Por qué write() retorna antes de que el dato esté en disco?

Hipótesis. write() solo copia el dato al page cache en RAM, y el kernel lo baja a disco después (writeback diferido). Escribí 200 MB con dd y miré Dirty y Writeback de /proc/meminfo antes y después de un sync, y los parámetros que gobiernan el writeback.

```
dd if=/dev/zero of=/tmp/a bs=1M count=200 status=none
grep -E '^(Dirty|Writeback):' /proc/meminfo
sync; grep -E '^(Dirty|Writeback):' /proc/meminfo
sysctl vm.dirty_expire_centisecs vm.dirty_writeback_centisecs vm.dirty_background_ratio
vm.dirty_ratio
```

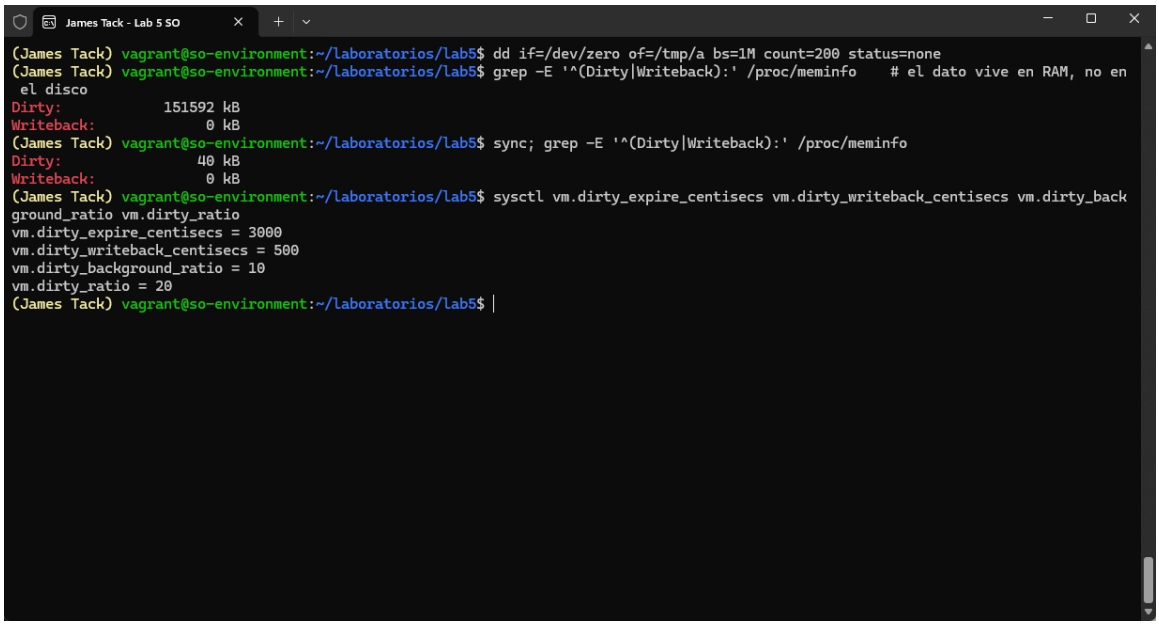


Figura 14. P1: justo después de dd hay 151 592 kB sucios en RAM; tras sync quedan 40 kB.

Resultado. dd terminó y 151 592 kB seguían marcados como Dirty, es decir, el comando "terminó" sin que el dato estuviera en disco. Un sync los bajó a 40 kB. Los parámetros dicen cuánto puede esperar un dato: vm.dirty_expire_centisecs = 3000 (30 s) y vm.dirty_writeback_centisecs = 500 (el hilo de flush despierta cada 5 s). Un dato recién escrito puede vivir hasta 30 segundos solo en RAM.

Para ver la diferencia a nivel de llamadas al sistema usé strace con dd conv=fsync. Los write() devuelven enseguida y la única llamada que garantiza persistencia es el fsync del final.

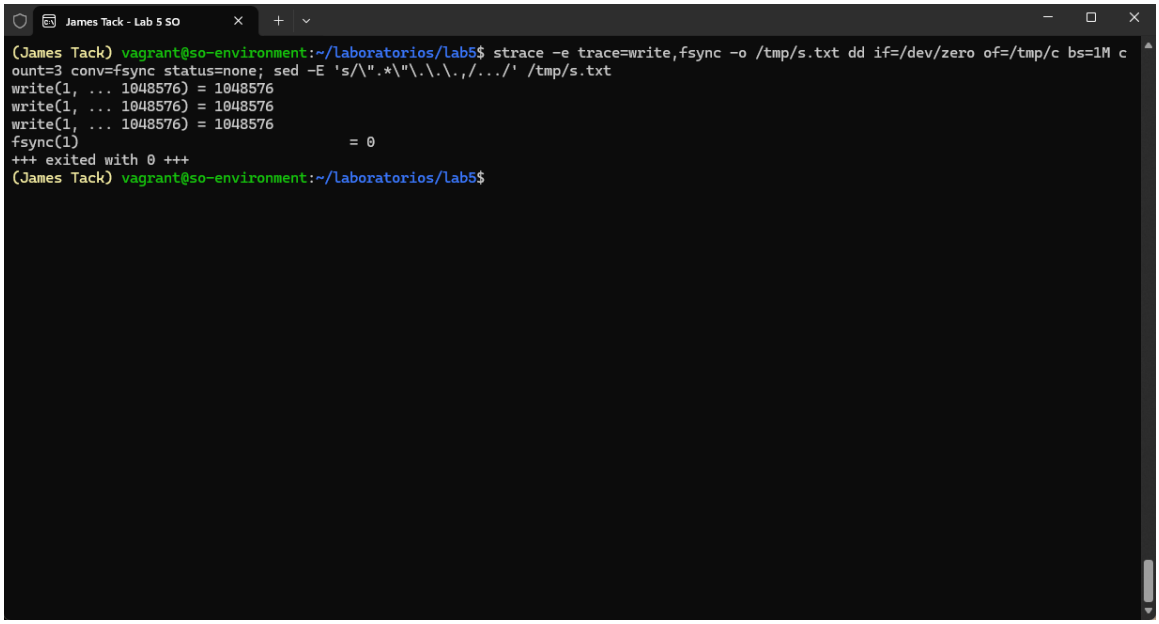


Figura 15. strace de dd conv=fsync: tres write() de 1 MiB y un fsync que devuelve 0 al final.

P2. Crear los archivos

El enunciado pide un punto de montaje /mnt/prueba. No tengo un disco extra, así que construí uno con una imagen de 256 MB formateada en ext4 y montada en loop. Así el experimento usa un sistema de archivos real con su journal y puedo analizarlo después con dumpe2fs. El mount confirma data=ordered, el modo por defecto de ext4.

```
dd if=/dev/zero of=disco.img bs=1M count=256 status=none && mkfs.ext4 -q -F disco.img &&
sudo mkdir -p /mnt/prueba && sudo mount -o loop disco.img /mnt/prueba && sudo chown vagrant
/mnt/prueba
mount | grep prueba; sudo dumpe2fs -h disco.img 2>/dev/null | grep -iE 'features|state'
```

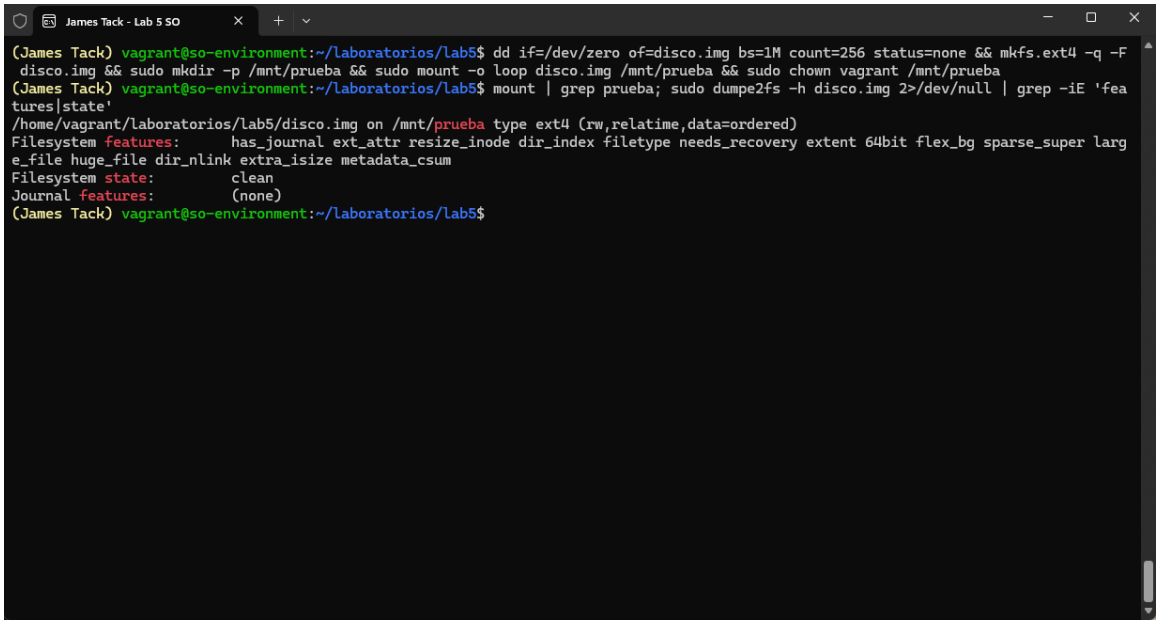


Figura 16. Imagen ext4 montada en /mnt/prueba con data=ordered, has_journal y estado clean.

Después escribí dos archivos de 64 MB con datos aleatorios. a.bin usa conv=fsync, o sea, dd llama a fsync antes de salir. b.bin no. Guardé el sha256 de cada uno en hashes.txt, que dentro de mi carpeta forcé a disco con sync hashes.txt para que la prueba no dependa de él. Al final consulté Dirty.

```
R=/mnt/prueba
dd if=/dev/urandom of=$R/a.bin bs=1M count=64 conv=fsync status=none
sha256sum $R/a.bin | tee ~/laboratorios/lab5/hashes.txt
dd if=/dev/urandom of=$R/b.bin bs=1M count=64 status=none
sha256sum $R/b.bin >> ~/laboratorios/lab5/hashes.txt; sync ~/laboratorios/lab5/hashes.txt;
cat ~/laboratorios/lab5/hashes.txt
grep '^Dirty:' /proc/meminfo
```

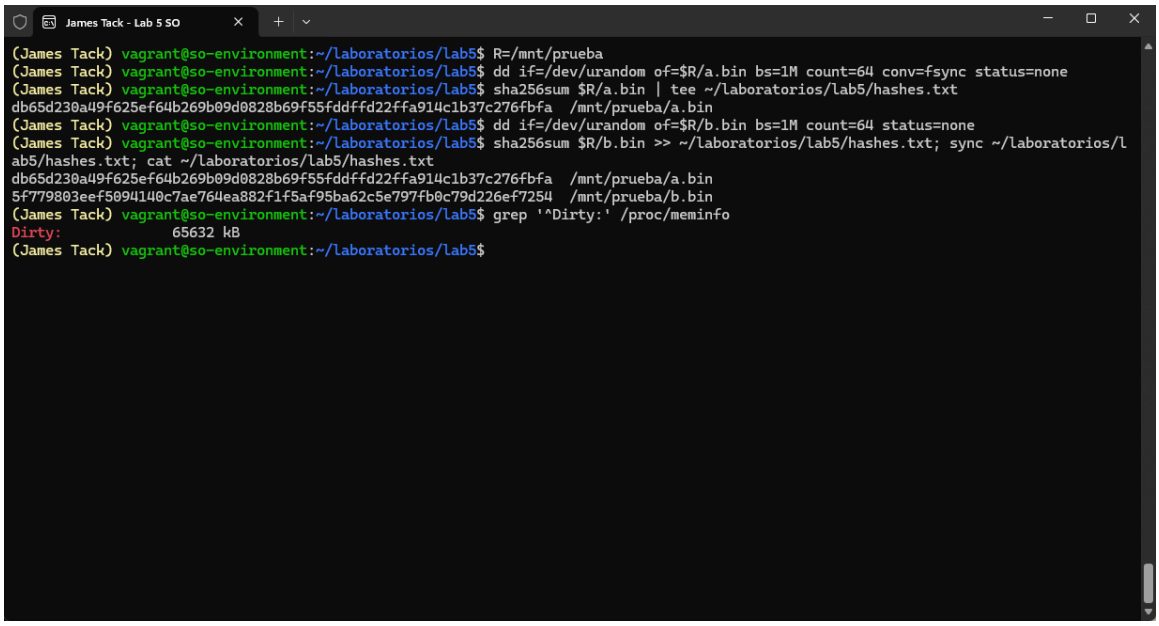


Figura 17. a.bin se escribe con fsync y b.bin sin fsync; Dirty muestra 65 632 kB pendientes.

Resultado. Los dos hashes quedaron guardados y Dirty marca 65 632 kB, casi exactamente los 64 MB de b.bin. El archivo con fsync ya está en disco y el otro vive solo en RAM.

P3. Provocar la caída

Con Dirty todavía en 65 MB envié la letra b a /proc/sysrq-trigger, que reinicia la máquina sin sincronizar ni desmontar nada. Es el equivalente a cortar la luz. Antes de la caída Dirty marcaba 65 540 kB.

```
grep ^Dirty: /proc/meminfo; echo b | sudo tee /proc/sysrq-trigger
```

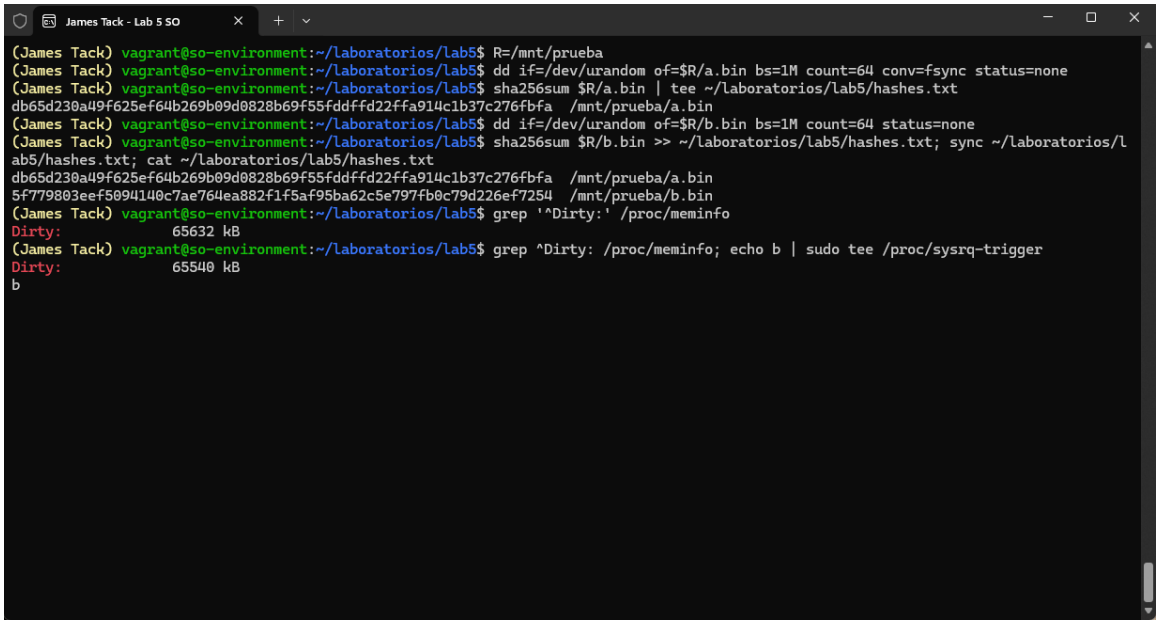


Figura 18. Caída dura: Dirty = 65540 kB y reinicio inmediato con sysrq b.

Después de la caída la VM se quedó colgada en el arranque, un problema que ya tenía en esta máquina, así que hice un reset con VBoxManage controlvm vm-sistemas-operativos reset. Es otra caída dura sobre la misma VM y no cambia el experimento.

P4. Tras el reinicio

Volví a entrar por ssh, monté la imagen otra vez (en loop, porque ya no existe el /dev/sdX del enunciado) y comparé los hashes guardados con sha256sum -c. También miré el dmesg y volví a abrir el sistema de archivos con dumpe2fs y debugfs.

```
cd ~/laboratorios/lab5; uptime; ls; cat hashes.txt
sudo mount -o loop disco.img /mnt/prueba && ls -l /mnt/prueba
sha256sum -c ~/laboratorios/lab5/hashes.txt
dmesg | grep -iE 'ext4|recovery' | tail
```

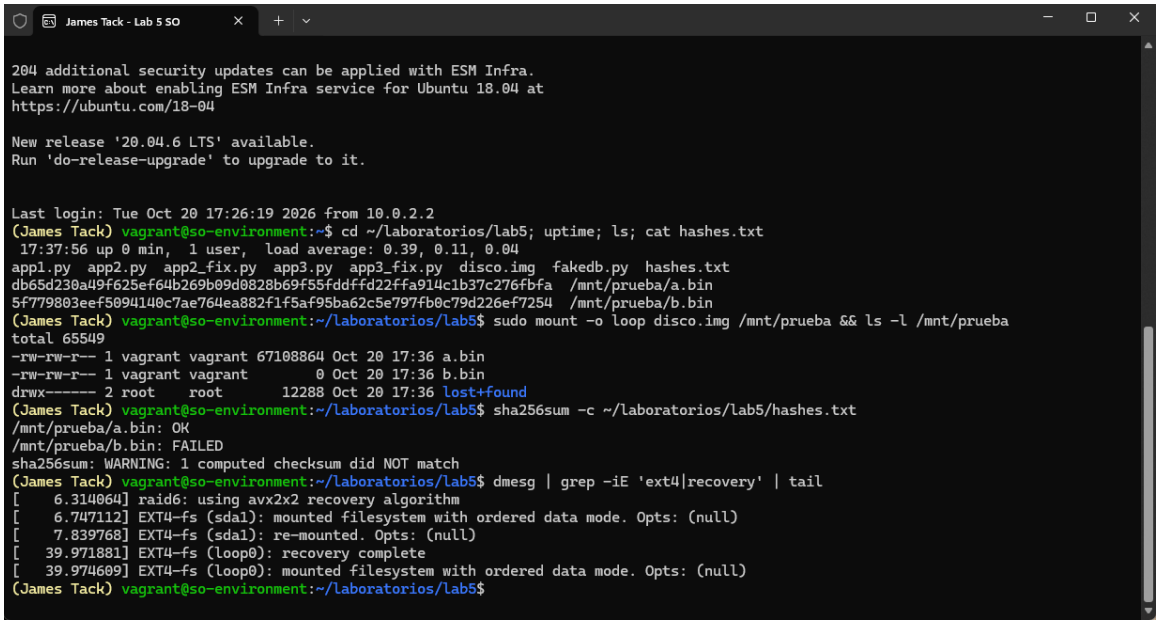


Figura 19. Tras el reinicio: a.bin OK; b.bin de 0 bytes con FAILED; ext4 hace "recovery complete".

Resultado. a.bin conserva sus 67 108 864 bytes y su hash coincide (OK). b.bin existe pero tiene 0 bytes, y su hash falla (FAILED). El dmesg muestra "EXT4-fs (loop0): recovery complete": ext4 reprodujo el journal al montar. El hash guardado en hashes.txt sobrevivió porque lo forcé a disco con sync antes de la caída.

```
sudo umount /mnt/prueba; sudo dumpe2fs -h disco.img 2>/dev/null | grep -iE 'filesystem
state|journal features|last mounted|mount count'
sudo debugfs -R 'ls -l /' disco.img 2>&1 | cut -c1-90
```

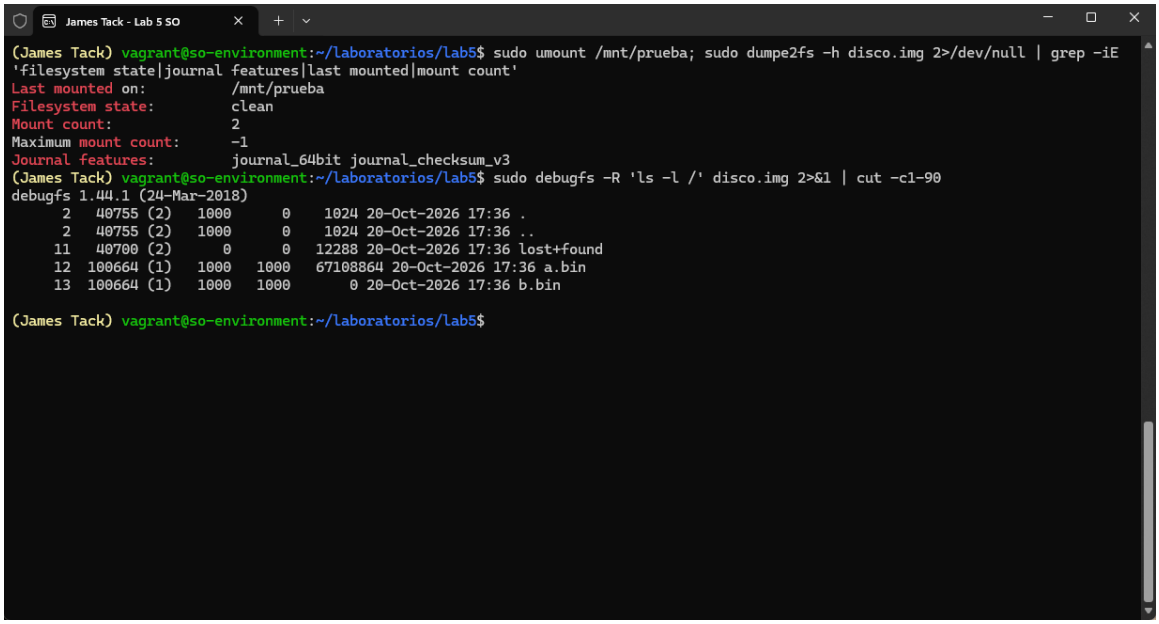


Figura 20. dumpe2fs: sistema de archivos clean con journal; debugfs: a.bin de 67108864 bytes y b.bin de 0.

El sistema de archivos quedó consistente (clean) tras la recuperación, y el listado de debugfs confirma el tamaño de cada archivo en el disco. Este es el punto importante: **ext4 conserva la consistencia, no la durabilidad de los datos que nadie pidió persistir.**

Análisis y runbook de durabilidad

write() solo escribe en el page cache, y el kernel hace el writeback diferido (hasta 30 s según vm.dirty_expire_centisecs). fsync() obliga a persistir en disco y a esperar a que el dispositivo termine, por eso MySQL confirma una transacción solo después de un fsync del log (innodb_flush_log_at_trx_commit = 1). En mi prueba, a.bin llegó entero porque dd llamó a fsync, y b.bin no llegó porque estaba entre los 65 MB sucios cuando cayó la máquina.

El modo data=ordered de ext4 explica el estado en que quedó b.bin. En ese modo ext4 solo registra en el journal los metadatos y obliga a escribir primero los datos de un archivo y después sus metadatos. Es el resultado esperable con la asignación retrasada de bloques (delalloc): el inodo de b.bin quedó en el journal con tamaño 0 porque sus datos nunca llegaron a asignarse en disco, y la recuperación lo dejó así: el archivo existe, el sistema de archivos está sano y el contenido se perdió. Si la caída hubiera interrumpido una escritura a medias, el resultado sería una torn write: una

página que quedó mitad nueva y mitad vieja. Por eso la confirmación de la aplicación tiene que depender de un fsync y no de que write() haya devuelto.

Síntoma	Comando	Lectura	Acción
Datos perdidos tras un crash	grep '^Dirty:' /proc/meminfo antes de caer	Dirty > 0 al momento de la caída (aquí 65 540 kB)	Exigir fsync / O_SYNC en la ruta de commit (o replicar)
Riesgo de pérdida en producción (alerta)	grep -E '^(Dirty|Writeback):' /proc/meminfo; sysctl vm.dirty_expire_centisecs	Dirty mayor de 64 MB sostenido durante más de 30 s (el valor de dirty_expire_centisecs)	Alertar; revisar si la ruta de commit llama a fsync y si el disco absorbe el writeback

La métrica es Dirty de /proc/meminfo y el umbral lo da el propio kernel: un dato sucio solo debería durar 30 segundos (vm.dirty_expire_centisecs = 3000). Si Dirty se queda alto más tiempo, el disco no sigue el ritmo de escritura. El umbral de 64 MB es el volumen que ya me dejó un archivo entero sin persistir en esta prueba, y se afina con la carga real de cada servidor.

Preguntas de anclaje

1. En P2, ¿por qué el error es EMFILE y no ENFILE? ¿Qué sysctl produciría ENFILE? Porque el bucle chocó con el soft limit del proceso (1024), que es RLIMIT_NOFILE, mientras el sistema tenía 1056 descriptores de 200 714 posibles. ENFILE (errno 23, "Too many open files in system") aparece cuando se agota el total del kernel, que fija fs.file-max.

2. ¿Por qué dos rutas al mismo archivo comparten el offset (mismo struct file) pero no se cierran juntas? Con exec 4>&3 el descriptor 4 es un duplicado del 3 y apunta a la misma entrada struct file del kernel. Por eso los dos muestran pos 11 en /proc/$$/fdinfo tras escribir "hola" y "mundo", y las escrituras por 3 y por 4 se continúan una a la otra. Cada descriptor es una entrada propia de la tabla del proceso, así que cerrar el 3 no cierra el 4: el archivo se mantiene abierto hasta que se cierre la última referencia, y escribí "otra" por el 4 después de cerrar el 3.

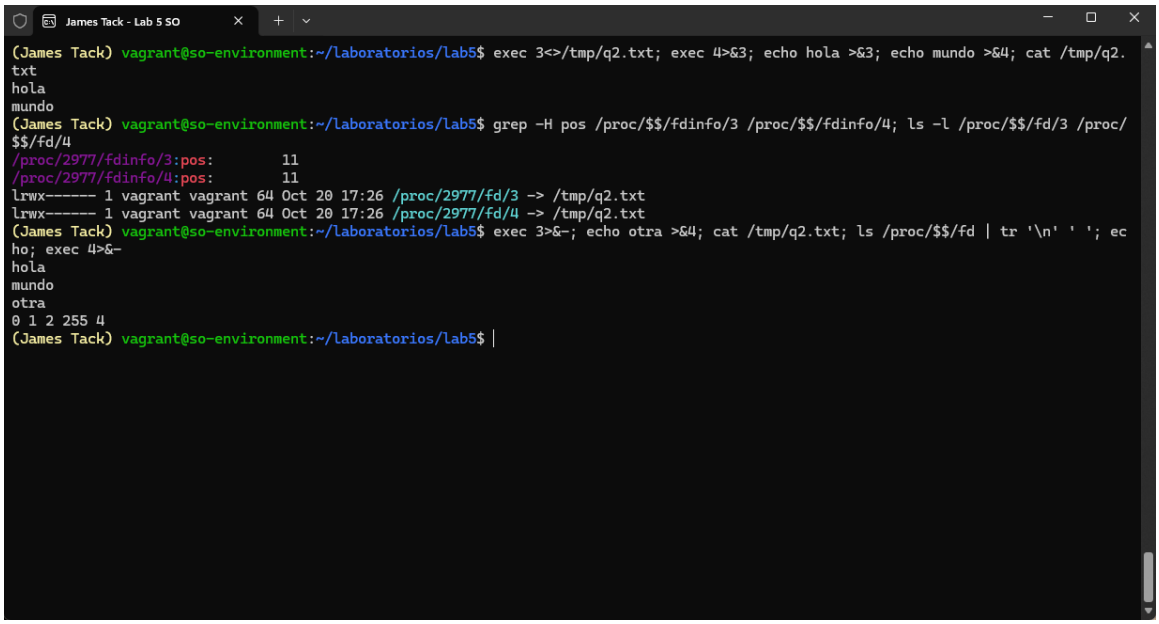


Figura 21. Los FDs 3 y 4 comparten posición (pos 11) y cerrar el 3 no cierra el 4.

3. En N3, ¿por qué el hash guardado sí sobrevive pero el archivo sin fsync puede no hacerlo? Porque hashes.txt lo forcé a disco con sync antes de la caída, y b.bin no. Sin ese sync el hash estaría en la misma situación que b.bin: sobrevive solo si el writeback ya lo había bajado, lo que ocurre a los 30 segundos como máximo.

4. Nombra la estructura del kernel detrás de cada síntoma. 502 → la tabla de descriptores de archivo del proceso (fdtable/struct file, limitada por RLIMIT_NOFILE y por fs.file-max en total). Pérdida de datos → el page cache, con sus páginas sucias (dirty pages) pendientes de writeback.

Lista de comprobación

Requisito del enunciado	Cumplido	Evidencia
N1 P1: FDs y límites del proceso, file-nr y file-max	Sí	Figura 2
N1 P2: EMFILE reproducido y diferenciado de ENFILE	Sí	Figura 3
N1 P3: fuga confirmada con strace	Sí	Figura 4
N1 P4: remediación aplicada y verificada	Sí	Figura 5
N1: runbook con comando, lectura y acción	Sí	Tabla de decisiones y runbook del 502
N2: método propio e identificación del culpable	Sí	Figuras 6 a 9

N2: remediación en 3 capas (código, proceso, kernel)	Sí	Figuras 10 a 13
N3: experimento write() contra fsync con caída dura	Sí	Figuras 14 a 20
N3: runbook de durabilidad con métrica y umbral	Sí	Tabla del análisis de N3
Preguntas de anclaje	Sí	Sección de preguntas, figura 21

Conclusiones

El 502 de Nginx con "24: Too many open files" no era un problema de red ni de CPU. Era un proceso que llegó al límite de 1024 descriptores y el kernel le negó el siguiente (EMFILE). Lo que más me ayudó fue medir antes de opinar: /proc/<pid>/limits y /proc/<pid>/fd bastaron para ver el techo y la tendencia, y strace -c sin lsof ni herramientas extra mostró la causa. Con los tres procesos del N2 comprobé que lo que importa es la tendencia, porque dos procesos con la misma cantidad de FDs en un instante pueden ser uno sano y uno con fuga.

La pérdida de datos del MySQL se explica con el page cache. Una escritura que "terminó" puede seguir hasta 30 segundos solo en RAM. En mi prueba un archivo sin fsync quedó con 0 bytes después de la caída, mientras que el archivo escrito con fsync quedó completo, y en los dos casos ext4 se recuperó sin errores. Es decir, el journal protege la estructura del sistema de archivos, no los datos que nadie pidió persistir.

Las dos fallas tienen la misma forma: un límite o un retraso del sistema operativo que la aplicación no toma en cuenta. La solución también es parecida, ya que consiste en que la aplicación cierre lo que abre y confirme solo lo que persistió, y en que el operador tenga una métrica y un umbral para detectarlo a tiempo.

Anexos

Anexo A. Comandos ejecutados

Paso	Dónde	Comando	Para qué
1	Windows	vagrant up	Encender la VM vm-sistemas-operativos
1	VM	ulimit -Sn; ulimit -Hn	Ver los límites de FDs del shell
P1	VM	grep 'open files' /proc/$P/limits; cat /proc/sys/fs/file-nr	Techo del proceso y contadores globales
P2	VM	ulimit -Sn 1024 + exec {fd}<>/dev/null	Provocar EMFILE
P3	VM	sudo strace -f -c -p $P	Contar openat contra close
P4	VM	ulimit -n 65535	Elevar el límite y verificar
N2	VM	ls /proc/$p/fd \| wc -l; ls -l /proc/$p/fd; ss -tn	Muestrear FDs, tipo y conexiones
N2	VM	systemctl show nginx -p LimitNOFILE; drop-in LimitNOFILE=65535	Capa de proceso
N2	VM	sysctl fs.file-max fs.nr_open	Capa de kernel
N3	VM	dd conv=fsync; grep '^Dirty:' /proc/meminfo	Mostrar el page cache y el fsync
N3	VM	mkfs.ext4 + mount -o loop	Sistema de archivos de prueba

N3	VM	echo b | sudo tee /proc/sysrq-trigger	Caída dura
N3	Windows	VBoxManage controlvm vm-sistemas-operativos reset	Reiniciar la VM colgada en el arranque
N3	VM	sha256sum -c hashes.txt; dumpe2fs -h; debugfs -R ls	Verificar tras la caída

Anexo B. Archivos del escenario

app1.py

```
import os, time
while True:
    try:
        with open("/etc/hosts") as f:
            f.read()
    except OSError as e:
        print("error:", e, flush=True)
    time.sleep(0.1)
```

app2.py

```
import os, time
while True:
    try:
        os.open("/etc/hosts", os.O_RDONLY)
    except OSError as e:
        print("error:", e, flush=True)
    time.sleep(0.1)
```

app3.py

```
import socket, time
pool = []
while True:
    try:
        pool.append(socket.create_connection(("127.0.0.1", 3306)))
    except OSError as e:
        print("error:", e, flush=True)
    time.sleep(0.1)
```

fakedb.py

```
import socket
srv = socket.socket()
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 3306)); srv.listen(2048)
conns = []
while True:
    c, _ = srv.accept()
    conns.append(c)
```

app2_fix.py

```
import os, time
while True:
```

```
    try:
        fd = os.open("/etc/hosts", os.O_RDONLY)
        try:
            os.read(fd, 1024)
        finally:
            os.close(fd)
    except OSError as e:
        print("error:", e, flush=True)
    time.sleep(0.1)
```

app3_fix.py

```
import socket, time
while True:
    try:
        with socket.create_connection(("127.0.0.1", 3306)) as s:
            pass
    except OSError as e:
        print("error:", e, flush=True)
    time.sleep(0.1)
```